SONIC PLAYWRIGHT FRAMEWORK

5-Minute Interview Explanation – Two Formats

Purpose: Prepare two interview-ready explanations of the same automation framework, one from the perspective of an engineer using an existing framework and the other from the perspective of an architect who designs and builds it.

Format 1: Explaining the Framework as a User

Role focus: Using the existing framework to automate scenarios, execute tests, validate results, and troubleshoot failures.

5-Minute Speaking Script

I have been working with the Sonic automation framework, which is built using Playwright and TypeScript. It provides a structured and reusable approach to automating application workflows.

The main objective of this framework is to reduce code duplication, improve maintainability, and make test execution more reliable.

Let me explain how I use this framework in my day-to-day automation activities.

First, I understand the application workflow and identify the test scenarios that need to be automated. For example, if I need to automate a login scenario, I identify the steps involved, such as navigating to the application, entering the username and password, clicking the login button, and verifying whether the login was successful.

Next, I use the existing framework structure to implement the test case. The framework follows the Page Object Model, so I maintain page-specific locators and actions in the corresponding page class instead of placing all the application logic directly inside the test.

For example, the login page class contains the username field, password field, login button, and methods related to login functionality.

The framework also provides reusable wrapper methods for common Playwright operations, such as clicking elements, filling text fields, interacting with dropdowns, and retrieving element information. I use these methods wherever possible instead of repeatedly writing the same low-level Playwright code.

This improves consistency and makes the test scripts easier to maintain.

The framework also uses fixtures to provide the required test setup and dependencies. I follow the existing fixture and configuration conventions when writing new tests, so that the tests integrate properly with the framework.

For test data, I use the available data utilities and supported data sources, such as JSON or CSV files, wherever applicable. For API-related scenarios, I can use the framework's API utilities and services rather than implementing the same functionality repeatedly.

Once the test is implemented, I execute it using the configured Playwright Test commands. I review the execution results and assertions to confirm whether the expected behavior matches the actual application behavior.

For debugging, I analyze the failure message, locator behavior, test data, and execution logs. I also use the available reporting and screenshot capabilities to investigate failures.

The framework supports HTML reporting and is integrated with GitHub Actions for CI/CD execution. This helps the team execute automated tests through the configured pipeline and review the results.

From a maintenance perspective, if a locator or page interaction changes, I update the corresponding page object or reusable method rather than modifying every test that uses it.

Overall, my responsibility as a framework user is to develop reliable test scenarios by following the existing framework standards, reuse available components, troubleshoot failures, and maintain the automation scripts as the application evolves.

Key Points to Emphasize

- Understand the framework structure and execution flow.
- Create test cases using the existing Page Object Model.
- Reuse wrapper methods, fixtures, utilities, and data-handling components.
- Validate application behavior using assertions.
- Debug failures using reports, logs, and available debugging capabilities.
- Maintain test scripts when application functionality changes.

Likely Follow-Up Question

Question: How do you add a new test case to the existing framework?

Suggested answer: I identify the scenario, reuse or create the required page object methods, follow the framework's fixtures and test-data conventions, add assertions, execute the test, and validate the report.

Format 2: Explaining the Framework as an Architect

Role focus: Designing the architecture, reusable components, execution flow, and maintainability.

5-Minute Speaking Script

I designed the Sonic automation framework using Playwright and TypeScript, with the objective of building a modular, reusable, maintainable, and scalable test automation solution.

Instead of writing individual automation scripts independently, I structured the framework into different components so that test logic, page interactions, configuration, test data, reusable utilities, and reporting responsibilities are separated.

Let me explain the architecture and the important design decisions behind the framework.

The first major component is the test layer. This layer contains the test scenarios, test steps, and assertions. Its responsibility is to define what needs to be tested without duplicating the implementation details of every page interaction.

The second component is the page object layer, which follows the Page Object Model. I separate page-specific locators and business actions into individual page classes. For example, a login page class contains the locators and methods required to perform login operations.

This separation makes maintenance easier because application-specific changes can generally be handled in the corresponding page class.

The next component is the Playwright wrapper layer. I created reusable wrapper methods over common Playwright operations, such as clicking, filling fields, handling dropdowns, retrieving text, and performing other frequently used interactions.

The purpose of the wrapper is to centralize common interaction logic and provide a consistent way for page objects to interact with application elements. Depending on the implementation, it can also centralize logging, screenshots, waits, or error handling.

From an object-oriented programming perspective, the framework uses classes, inheritance, encapsulation, and abstraction where appropriate. For example, page classes can extend a common base or wrapper class to reuse shared behavior instead of duplicating it across multiple pages.

The framework also uses Playwright fixtures to manage test dependencies and setup. I keep configuration separate so that execution settings, browser options, timeouts, and environment-specific values can be managed centrally.

For test data management, I organized reusable utilities to work with supported formats such as JSON and CSV. Interfaces help define the expected structure of TypeScript data, while Faker can generate dynamic test data when required.

I also included supporting components for API automation, token generation, database interactions, Excel or request utilities, and verification-related operations where the test scenario requires them. This allows the framework to support more than UI-only automation.

For execution and reporting, I use Playwright Test's execution capabilities and the framework's reporting setup. The framework is also integrated with GitHub Actions, enabling automated test execution through the configured CI/CD pipeline.

The overall execution flow is straightforward: the test starts through Playwright Test, obtains the required setup through the fixture and configuration layers, invokes methods from the relevant page object, and performs application interactions through the reusable wrapper methods. The test then validates the expected outcome using assertions, and the execution results are made available through the reporting process.

A key design consideration was maintainability. I wanted common functionality to be implemented once and reused across multiple test scenarios. This reduces duplication and makes framework enhancements easier.

I also considered scalability by keeping responsibilities separated, so additional page objects, test cases, test data, and supporting utilities can be introduced without unnecessarily changing unrelated components.

Overall, my responsibility as the framework architect was to define the structure, establish reusable design patterns, separate concerns, implement common utilities, and provide a consistent foundation that allows the automation team to develop, execute, and maintain tests efficiently.

Key Points to Emphasize

- Architecture: Separate test, page, wrapper, utility, configuration, data, and reporting responsibilities.
- Design pattern: Page Object Model.
- Reusability: Common Playwright actions are centralized in wrapper methods.
- OOP concepts: Classes, encapsulation, inheritance, and abstraction where implemented.
- Test management: Playwright Test and fixtures.
- Data management: JSON, CSV, TypeScript interfaces, and Faker as applicable.
- Integration: API utilities, supporting services, reporting, and GitHub Actions.
- Design goals: Maintainability, scalability, consistency, and reduced code duplication.

Likely Follow-Up Question

Question: Why did you create a wrapper class instead of directly using Playwright methods everywhere?

Suggested answer: The wrapper centralizes common operations, reduces repetitive code, promotes consistency, and provides a single place to enhance shared interaction behavior. I would mention specific capabilities such as logging or error handling only if they are implemented in the actual wrapper.

Quick Comparison: User vs. Architect

Area	Framework User	Framework Architect
Main focus	Writing and maintaining tests	Designing and maintaining the framework
Page Object Model	Uses existing page classes	Defines the page object structure
Wrapper methods	Calls reusable methods	Designs and implements reusable methods
Fixtures/configuration	Follows existing conventions	Defines setup and configuration strategy
Test failures	Debugs individual failures	Improves framework-level diagnostics and reliability
Changes	Updates tests and page objects	Evaluates architecture and shared-component changes
Ownership	Test automation delivery	Framework design and technical direction

Interview Recommendations

- If asked, “Explain your automation framework,” use the architect version only if you genuinely designed the framework.
- If asked, “How do you use the existing framework in your project?”, use the user version.
- If you contributed to the framework but did not build every component, clearly distinguish the components you personally designed from the existing capabilities.
- Be ready to explain the wrapper class, base class, fixtures, POM structure, and execution flow with a small code example.